WEB APPLICATION ATTACK PROJECT #4

Tarannum CHOUDHARY • IMAD ALI SYED • TEJVEER WARAICH

SUMMARY



- Designed one dynamic Django app with "VULNERABLE MODE" toggle.
- Demonstrated six real-world web vulnerabilities:
  - SQL Injection
  - Cross-Site Scripting (XSS)
  - Directory Traversal
  - Parameter Tampering
  - Cookie Poisoning
  - Web Crawling
- Showed how to secure the application when "VULNERABLE MODE" is OFF.

BACKGROUND

Why Web Apps Are Attacked
Web apps are exposed to the internet = constant target.

Attackers exploit flaws in user input handling, session management, and access controls.

Developers must "think like hackers" to build secure systems.











TYPES OF ATTACKS:

SQL Injection (SQLi)

- Malicious SQL code injected via inputs.
- Impact: Bypass login, steal data, delete databases.
- Demo: ' OR '1'='1
- Defense: Always use parameterized queries (Django's authenticate()).

Cross-Site Scripting (XSS)

- JavaScript injected into pages seen by others.
- Impact: Cookie theft, session hijacking.
- Demo: alert("Hacked!")
- Defense: Escape output by default, sanitize inputs.



TYPES OF ATTACKS:

Directory Traversal
- Manipulating file paths to access unauthorized files.
- Impact: Leaking sensitive files like settings.py.
- Demo: ../../myproject/settings.py
- Defense: Strict input validation, whitelisted file access.

Parameter Tampering
- Manipulating URL parameters to escalate privileges.
- Impact: Become admin by changing ?role=admin
- Demo: Modify URL in Profile page.
- Defense: Server-side validation of user roles.



TYPES OF ATTACKS:

Cookie Poisoning
- Editing cookies to gain unauthorized access.
- Impact: Impersonate admin or other users.
- Demo: Intercept cookie, change user role.
- Defense: Signed cookies, secure session management.

Web Spidering
- Automated tools explore all accessible pages, links, and parameters on a website.
- Impact: Discover hidden files, sensitive directories, admin panels.
- Demo: Use Burp Suite Spider or OWASP ZAP to find hidden URLs like /admin/ or /files/.
- Defense: Block sensitive paths in robots.txt, secure folders with authentication, disable directory listing.

TOOLS AND SYSTEM



- Django Web Framework
- Python 3.13
- SQLite3 Database
- Burp Suite Community Edition
- VSCode
- System: Windows 10, 8GB RAM, Intel i5 Processor

EXPERIMENTAT



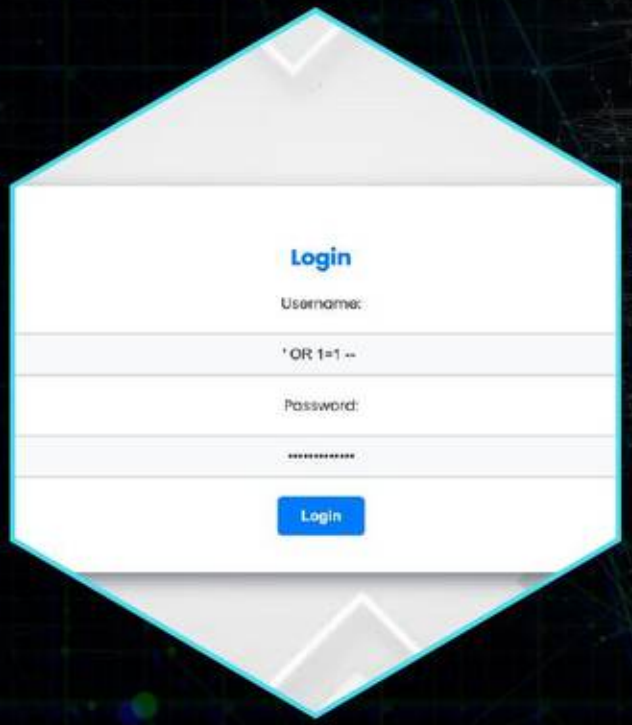


- SQL Injection at login page.
- XSS via comment posting.
- Directory traversal in file viewer.
- Role tampering in profile page.
- Cookie poisoning and Web Spidering with Burp Suite.

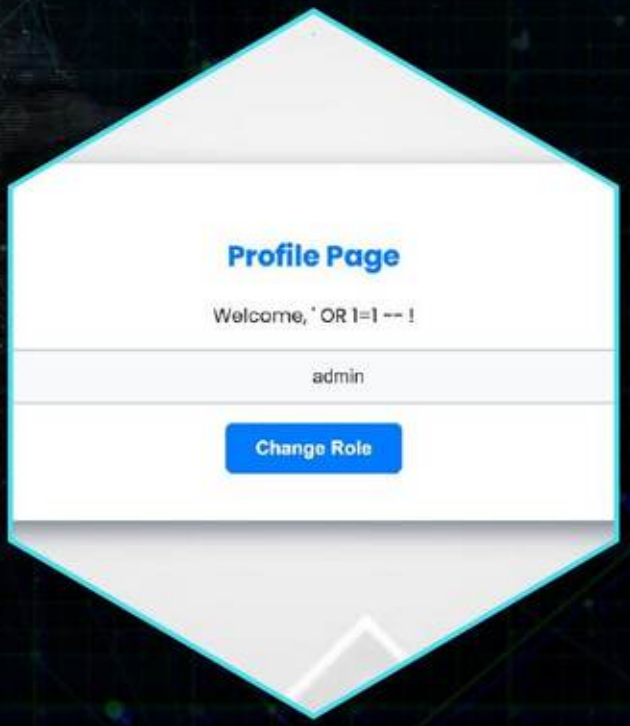
Flowchart:

- Visit page → Input attack → Exploit succeeds (vulnerable) / blocked (secure)

RESULTS:


Login
Username:
' OR 1=1 --
Password:
Login

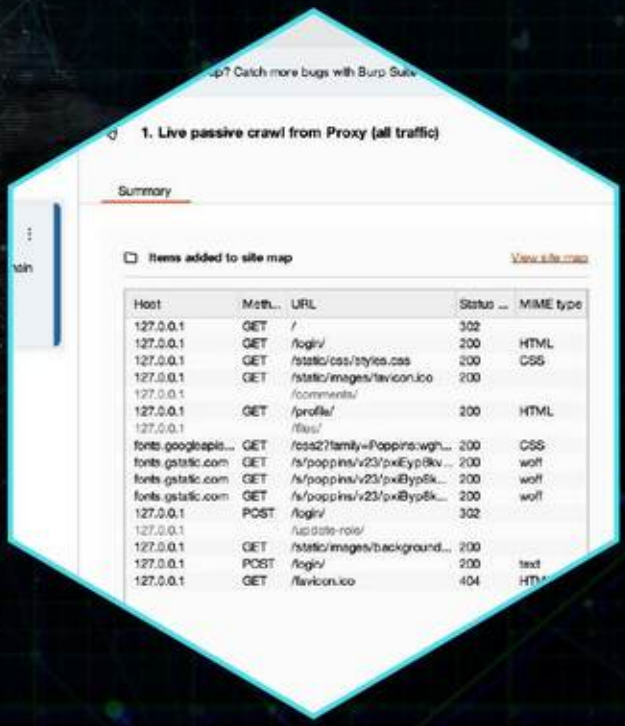

127.0.0.1:8000 says
Hacked!


Profile Page
Welcome, ' OR 1=1 -- !
admin
Change Role

RESULTS:


File Viewer
Enter Filename:
../../myproject/settings.py
View
[Errno 21] Is a directory: '/Users/imadalisyed/Documents/NWS/myproj


Target
Proxy
Intruder
HTTP history
WebSockets history
Intercept on
Forward
Time
Type
Direction
Method
URL
20:19:43 27 Ap...
HTTP
Request
GET


1. Live passive crawl from Proxy (all traffic)
Summary
Items added to site map
Host
URL
Status
MIME type

DISCUSSION & CONCLUSION

- Vulnerable mode: All attacks succeeded.
- Secure mode: All attacks blocked.

Takeaway:

- Security isn't one setting — it's layers of protection.
- Always validate inputs, encode outputs, and secure sessions.
- Web developers must "assume they are under attack" daily.



THANK YOU

